

Tutorial: Explorative Datenanalyse (EDA)

Explorative Datenanalyse (EDA) bedeutet, einen unbekannten Datensatz systematisch zu untersuchen, um seine wichtigsten Eigenschaften, Muster, Auffälligkeiten und Zusammenhänge zu verstehen – bevor man mit Modellierung oder tieferer Analyse beginnt. Dieses Tutorial zeigt EDA anhand eines Energie-Datensatzes mit Python.

Voraussetzungen

- Zugriff auf oder Berechtigung zum Erstellen von Compute-Ressourcen.
- Optional: Grundkenntnisse zu Genie Code, dem KI-Assistenten, der bei der automatischen Code-Erzeugung hilft.

Verwendete Python-Bibliotheken

- **NumPy** – numerische Berechnungen
- **pandas** – Datenmanipulation in Tabellenform
- **Plotly** – interaktive Diagramme
- **Matplotlib** – statische Diagramme

Schritt 1–2: Datensatz laden und Notebook anlegen

Der Beispieldatensatz (Energieverbrauch, `owid-energy-data.csv` von Kaggle) wird per Drag-and-drop in den Workspace importiert. Anschließend wird ein neues Python-Notebook erstellt und die Datei geladen:

```
import numpy as np
import pandas as pd
df = pd.read_csv('/Workspace/Users/.../owid-energy-data.csv')
```

Schritt 3: Daten verstehen

Mit `display(df)` lässt sich die Tabelle (über 10.000 Zeilen) interaktiv ansehen. Wichtige pandas-Befehle zur ersten Einschätzung:

- `df.shape` – Anzahl Zeilen und Spalten
- `df.dtypes` – Datentyp jeder Spalte
- `df.describe()` – Kennzahlen wie Mittelwert, Standardabweichung, Perzentile für numerische Spalten

Zusätzlich bietet Databricks ein eingebautes **Datenprofil** (über + → **Data Profile** neben der Ergebnistabelle): Es erzeugt automatisch Kennzahlen und Histogramme für alle Spalten, alternativ programmatisch über `dbutils.data.summarize()`.

Schritt 4: Daten bereinigen

Duplikate entfernen

```
duplicate_rows = df.duplicated().sum()
df = df.drop_duplicates()
df = df.loc[:, ~df.columns.duplicated()]
```

Fehlende Werte behandeln

```
df = df.fillna(0)
```

Ersetzt fehlende Werte (NaN, "Not a Number") durch 0, damit weitere Berechnungen möglich sind.

Datumsformate vereinheitlichen

```
df['year'] = pd.to_datetime(df['year'], format='%Y', errors='coerce').dt.year
```

Statt jeden Bereinigungs-schritt selbst zu programmieren, kann man Genie Code mit einfachen Aufforderungen wie "Prüfe, ob df doppelte Zeilen enthält, gib sie aus und lösche sie" nutzen – der KI-Assistent generiert den passenden Code.

Schritt 5: Eingebaute Tabellenfunktionen nutzen

`display(df)` bietet direkt in der Oberfläche eine Suche (Lupensymbol), Filter (Filtersymbol, teils per Genie Code automatisch generierbar) und die Möglichkeit, Diagramme über **+** → **Visualization** zu erstellen.

Schritt 6: Analyse und Visualisierung mit Python-Bibliotheken

Das Tutorial zeigt mehrere Beispielanalysen mit `plotly.express` (als `px` importiert):

- **Top-10-Emittenten 2000–2022:** Liniendiagramm der Treibhausgas-Emissionen der zehn größten Verursacherländer im Zeitverlauf. Ergebnis: Die Emissionen stiegen bei den meisten Ländern, nur wenige zeigten leichte Rückgänge.
- **Emissionen nach Weltregion:** Balkendiagramm je Kontinent. Ergebnis: Asien führt, Ozeanien/Südamerika/Afrika liegen am niedrigsten.
- **Anteil erneuerbarer Energien:** Ein neues Verhältnis `renewable_share = erneuerbare / Gesamtverbrauch` wird berechnet und für die zehn führenden Länder als Liniendiagramm dargestellt. Norwegen und Island liegen über 50 %.
- **Streudiagramm Emissionen vs. erneuerbare Energie:** Zeigt, dass ein höherer Anteil erneuerbarer Energien bei den größten Emittenten meist mit höherem Gesamtverbrauch (und damit höheren Emissionen) einhergeht – außer in Nordamerika, wo die Emissionen trotz steigendem Anteil erneuerbarer Energien stabil blieben.
- **Prognose des globalen Energieverbrauchs:** Mit der Bibliothek `statsmodels` wird ein **ARIMA-Modell** (ein statistisches Prognoseverfahren für Zeitreihen) trainiert und der Verbrauch für die nächsten 10 Jahre vorhergesagt, dargestellt mit Matplotlib. Ergebnis: Der globale Energieverbrauch wird laut Modell weiter steigen.

Fertiges Beispiel-Notebook

Ein komplettes Beispiel-Notebook "EDA mit globalen Energiedaten" steht zum direkten Öffnen oder Importieren bereit.

Wichtige Erkenntnisse

- Datenbereinigung (Duplikate, fehlende Werte, Formate) ist die Voraussetzung für aussagekräftige Analysen.
- Visualisierungen machen Muster und Zusammenhänge sichtbar, die in reinen Zahlenkolonnen leicht übersehen werden.

- Databricks bietet sowohl eingebaute Tabellenfunktionen (Filter, Suche, Profiling) als auch volle Flexibilität durch Python-Bibliotheken.

Nächste Schritte

Empfohlen werden: weitere Visualisierungsbeispiele, der eingebaute Debugger zur Fehlerbehebung, das Teilen von Notebooks im Team sowie das Erstellen von Dashboards für Stakeholder.

Quelle: <https://docs.databricks.com/aws/en/notebooks/eda-tutorial>

Stand: 2026-08-06